



CS 540 Introduction to Artificial Intelligence **Reinforcement Learning**

University of Wisconsin
Fall 2025 Section

Announcem

- **Homework:**
 - HW8 due on Friday November 2
 - HW9 will be released also on Fri

- Class roadmap:

Introduction
Learning

Reinforcem

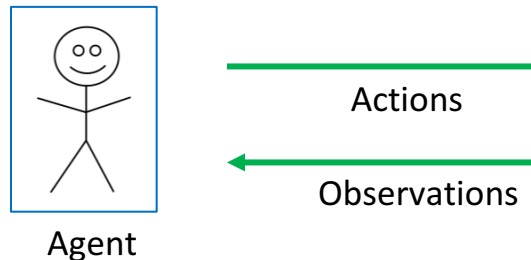
Thanksgivin

Outline

- Introduction to reinforcement
 - Basic concepts, mathematical formalism
- Learning policies
 - Q-learning, action-values, exploration

Back to Our Gene

We have an **agent** **interacting** w



- Agent receives a reward based on s
 - **Goal:** maximize reward / utility (\$\$\$)
 - Note: **data** consists of actions & obser
 - Compare to unsupervised learning and sup

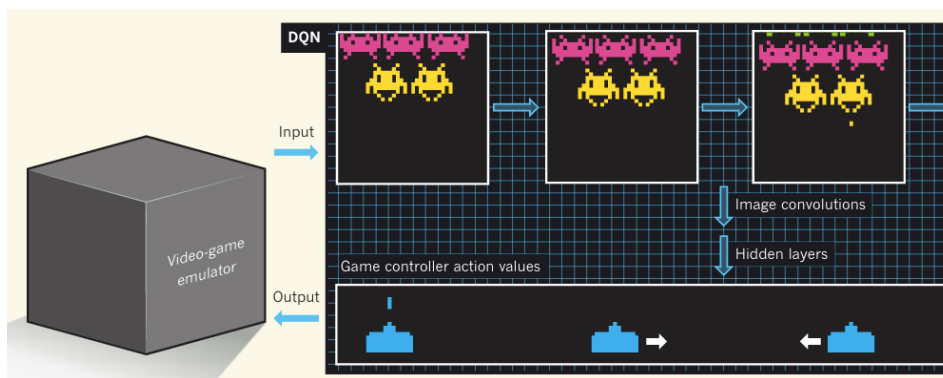
Examples: Gamep

AlphaZero:



Examples: Video G

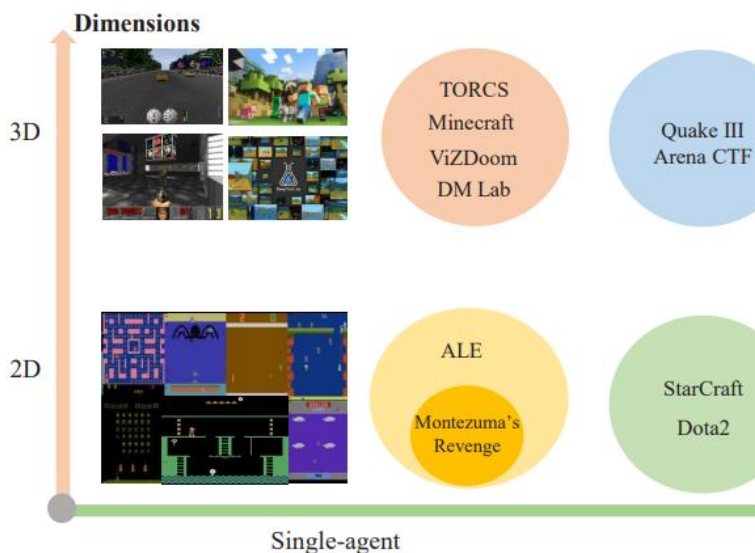
Pong, Atari



Mnih et al, "Human-level control through deep reinforcement learning"

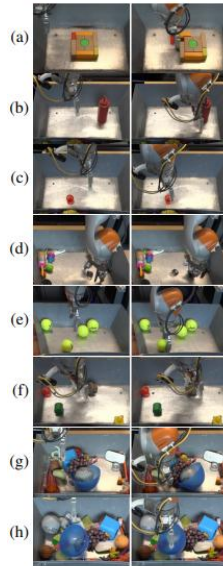
Examples: Video Games

Minecraft, Quake, StarCraft, and



Examples: Ro

Training robots to perform tasks

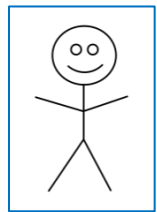


Building The Theor

Basic setup:

- Set of states, S
- Set of actions, A
- Information: at time t , observe state
- Agent makes choice $a_t \in A$. State ch

Goal: find a map from **states to action**



Agent



A “policy”

Markov Decision Process

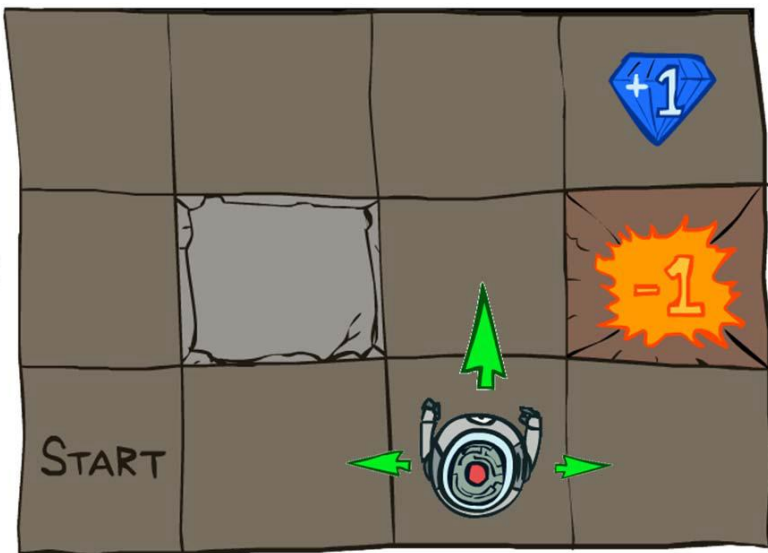
The formal mathematical model

- **State set** S . Initial state s_0 . **Action set** A
- **Reward function**: $r(s_t)$
- **State transition model**: $P(s_{t+1} | s_t, a_t)$
 - Markov assumption: transition probability depends only on current state and action, and not earlier history (previous actions and states)
- More generally: $r(s_t, a_t)$, potential rewards
- **Policy**: $\pi(s) : S \rightarrow A$ action to take in state s

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2$$

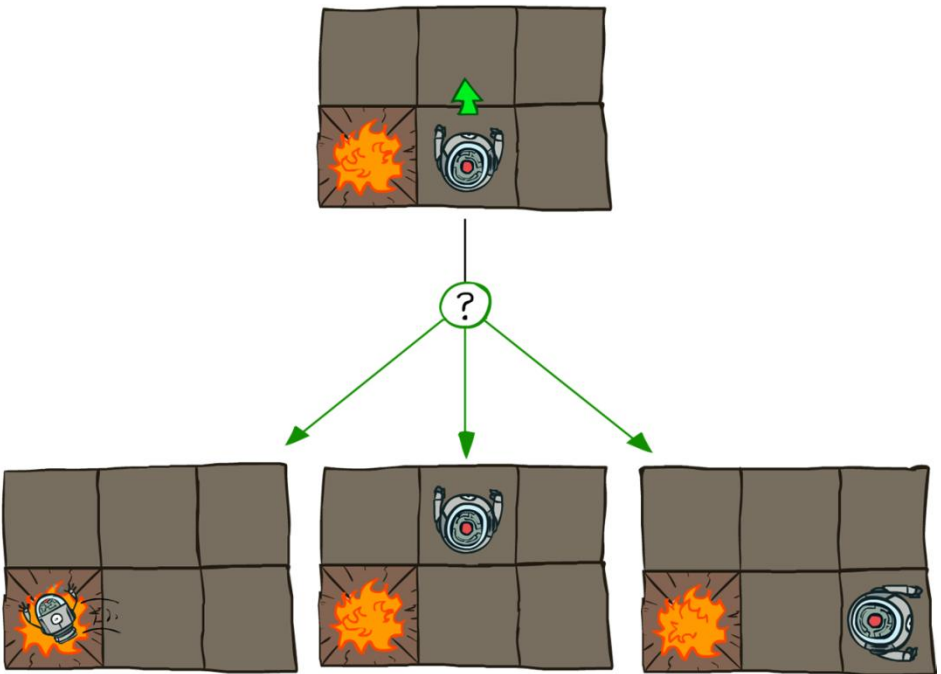
Example of MDP:

Robot on a grid; goal: find the be



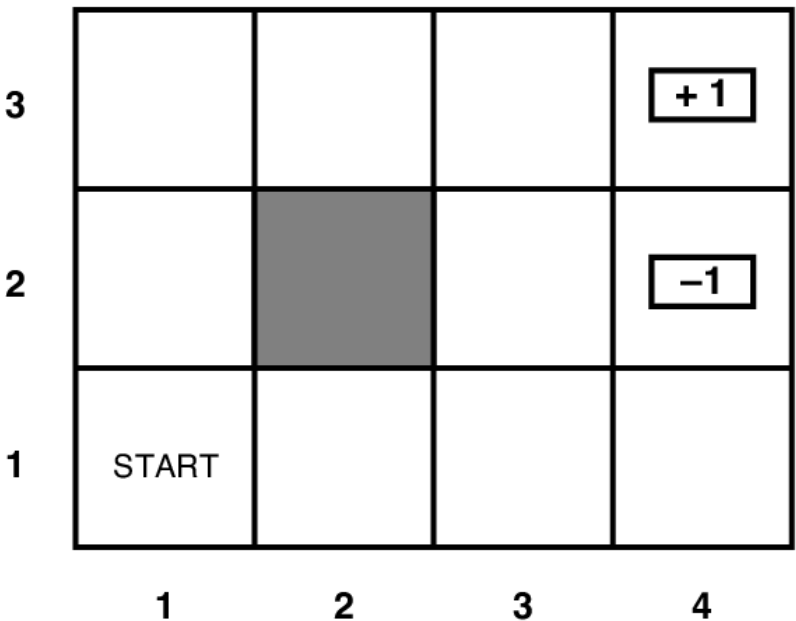
Example of MDP:

Note: (i) Robot is unreliable (ii)



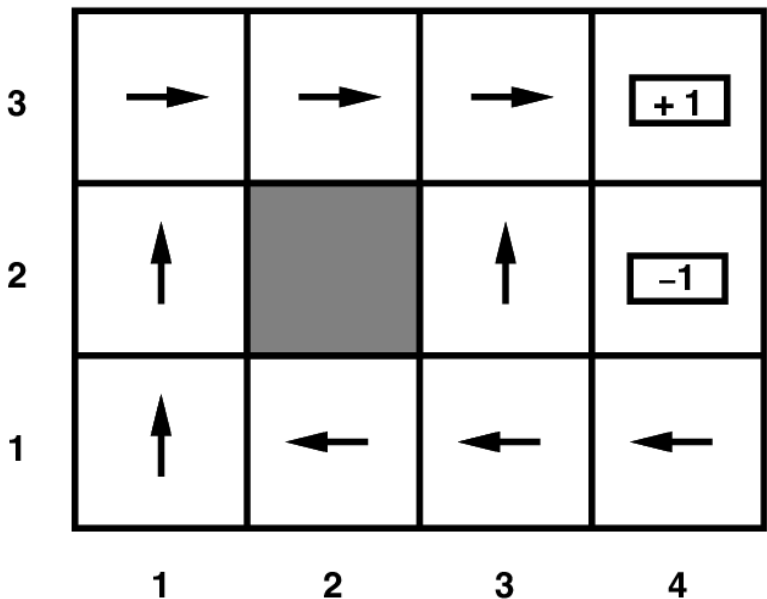
Grid World Abs

Note: (i) Robot is unreliable (ii)



Grid World Opti

Note: (i) Robot is unreliable (ii)



Back to MDP

The formal mathematical model

- **State set** S . Initial state s_0 . **Action set** A .
- **State transition model:** $P(s_{t+1} | s_t, a_t)$
 - Markov assumption: transition probability depends only on the current state and action, and not previous actions or states.
- **Reward function:** $r(s_t, a_t)$
- **Policy:** $\pi(s) : S \rightarrow A$ action to take in state s .

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2$$

Reinforcement Learning

Credit-assignment:

- May take many actions before reward is important?
- Example: You study 15 minutes a day all day. At the final exam, you eat a bowl of yogurt. You wonder if the studying or the yogurt that led to the reward.

Exploration vs. Exploitation:

- Transition probabilities and reward may change over time.
- Should you keep trying actions that led to low reward, or try actions that might lead to even more reward?

Break & C

Q 1.1 Which of the following statements

- A. The reward function must output
- B. The policy maps states to actions
- C. The probability of next state can be calculated from previous states
- D. The solution of MDP is to find a policy that maximizes cumulative rewards

Break & C

Q 1.1 Which of the following statements

- A. The reward function must output
- B. The policy maps states to actions
- **C. The probability of next state can be calculated from previous states**
- D. The solution of MDP is to find a policy that maximizes cumulative rewards

Break & C

Q 1.1 Which of the following statement about

- A. The reward function must output a scalar (**compare**)
- B. The policy maps states to actions (**True: take for each state**).
- C. The probability of next state can depend (**False: Markov assumption**).
- D. The solution of MDP is to find a policy that maximizes rewards (**True: want to maximize rewards**)

Defining the Opti

For policy π , **expected utility** over sequences from s_0 produced by

$$V^\pi(s_0) = \sum_{\text{sequences starting from } s_0} P(s_0, \pi)$$

Prob
w



Called the **value function** (for π ,

Discounting R

One issue: these are possibly inf
Convergence?

- Solution: discount future rewa

$$U(s_0, s_1 \dots) = r(s_0) + \gamma r(s_1) + \gamma^2$$

- Discount factor γ between 0 and 1
 - Set according to how important
 - Note: has to be less than 1 for c

From Value to

Now that $V^\pi(s_0)$ is defined, wh

- First, let π^* be the **optimal** policy for maximizing expected utility.
- What's the expected utility following π^* ?
 - Specifically, action a in state s ?

$$\sum_{s'} P(s'|s, a) V^*(s')$$

All the states we
could go to

Transition probability

Slight Problem

Now we can get the optimal policy

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a)$$

- So we need to know $V^*(s)$ (and $Q^*(s, a)$)
 - But it was defined in terms of the optimal policy
 - So we need some other approach
 - Instead, learn about the utility

Bellman Equ

Let's walk over one step for the v

$$V^*(s) = r(s) + \gamma \max_a \sum_{s'} P(s'|s,a) V^*(s')$$



Current state
reward

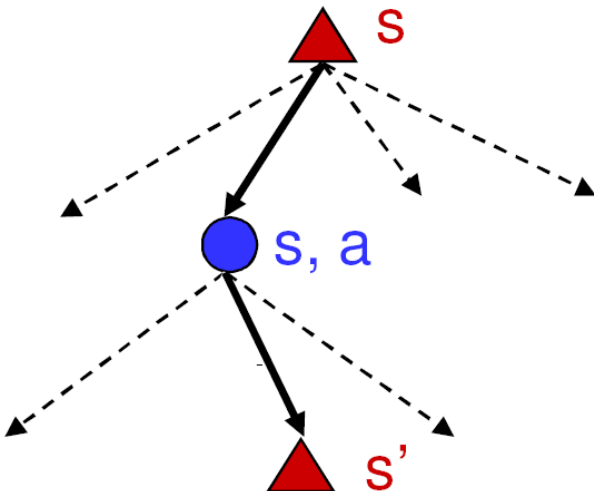
Disc
futu

- Richard Bellman: inventor of dynam

Bellman Equ

Let's walk over one step for the

$$V^*(s) = r(s) + \gamma V^*(s')$$



Current
reward

Value Itera

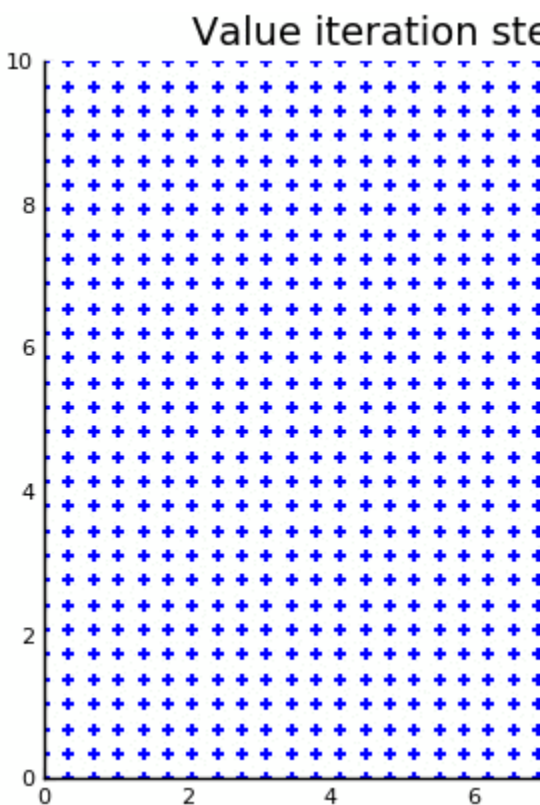
Q: how do we find $V^*(s)$?

- Why do we want it? Can use it to g
- Know: reward $r(s)$, transition proba
 - Knowing r and P is the “planning” prob
 - estimated from interactions : “reinfor
- Also know $V^*(s)$ satisfies Bellman e

A: Use the property. Start with V

$$V_{i+1}(s) = r(s) + \gamma \max_a \sum$$

Value Iteration



Break & C

Q 2.1 Consider an MDP with 2 states $\{A, B\}$ a state and “**move**” to other state. Let **r** be the 1, **r**(B) = 0. Let γ be the discounting factor. Let “always move” policy). What is the value fun

- A. 0
- B. $1 / (1 - \gamma)$
- C. $1 / (1 - \gamma^2)$
- D. 1

Break & C

Q 2.1 Consider an MDP with 2 states $\{A, B\}$ a state and “**move**” to other state. Let **r** be the 1, **r**(B) = 0. Let γ be the discounting factor. Let “always move” policy). What is the value fun

- A. 0
- B. $1/(1-\gamma)$
- **C. $1/(1-\gamma^2)$**
- D. 1

Break & C

Q 2.1 Consider an MDP with 2 states $\{A, B\}$ a state and “**move**” to other state. Let r be the 1, $r(B) = 0$. Let γ be the discounting factor. Let “always move” policy). What is the value fun

- A. 0
- B. $1/(1-\gamma)$
- **C. $1/(1-\gamma^2)$** (States: A,B,A,B,... rewa
- D. 1

Q-Learning

- Our **next** reinforcement learning
- Does not require knowing r or P .
form: $\{(s_t, a_t, r_t, s_{t+1})\}$.
- Learns an action-value function Q
expected value of taking a in state s
 - Note: $V^*(s) = \max_a Q^*(s, a)$
- Optimal policy is formed as $\pi^*(s)$

The $Q^*(s,a)$ f

- Starting from state s , perform action a . THEN follow the opt

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s' | s, a) Q^*(s', a)$$

- Equivalent to

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s' | s, a) Q^*(s', a)$$

Q-Learning Itc

How do we get $Q(s, a)$?

- Iterative procedure

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r(s_t) + \gamma V(s_t) - Q(s_t, a_t)]$$

Learning rate



Idea: combine old value and new esti

Note: We are using a policy to take ac
estimated Q!

Q-Learning

Estimate $Q^*(s, a)$ from data $\{(s, a, r, s')\}$

1. Initialize $Q(.,.)$ arbitrarily (e.g. 0)
 1. Except terminal states $Q(s_{\text{term}}, a)$
2. Iterate over data until $Q(.,.)$ converges

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(r_t + \max_{a'} Q(s_t, a'))$$



Learning rate

Exploration Vs. Exploitation

General question!

- **Exploration:** take an action with unknown consequences
 - **Pros:**
 - Get a more accurate model of the environment
 - Discover higher-reward states than the ones currently known
 - **Cons:**
 - When exploring, not maximizing your utility
 - Something bad might happen
- **Exploitation:** go with the best strategy found so far
 - **Pros:**
 - Maximize reward as reflected in the current utility
 - Avoid bad stuff
 - **Cons:**
 - Might prevent you from discovering the true optimum

Q-Learning: ε -Greedy

Getting data with both **explorati**

- With probability ε , take a random action
the highest (current) $Q(s, a)$ value.

$$a = \begin{cases} \operatorname{argmax}_{a \in A} Q(s, a) \\ \text{random } a \in A \end{cases}$$

Q-learning Algorithm

Input: step size α , exploration probability ϵ

1. set $Q(s,a) = 0$ for all s, a .
2. For each episode:
 3. Get initial state s .
 4. While (s not a terminal state):
 5. Perform $a = \epsilon$ -greedy(Q, s), receive r
 6. $Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(r + \max_{a'} Q(s, a'))$
 7. $s \leftarrow s'$
 8. End While
9. End For

Break & C

Q 3.1 For Q learning to converge to the

- A. Visit every state and try every a
- B. Perform at least 20,000 iteration
- C. Re-start with different random i
- D. Prioritize exploitation over expl

Break & C

Q 3.1 For Q learning to converge to the

- **A. Visit every state and try every a**
- B. Perform at least 20,000 iterations
- C. Re-start with different random initialization
- D. Prioritize exploitation over exploration

Break & C

Q 3.1 For Q learning to converge to the true

- **A. Visit every state and try every action**
- B. Perform at least 20,000 iterations. (Not a particular problem, not a general constant necessary in general).
- C. Re-start with different random initial values (not necessary in general).
- D. Prioritize exploitation over exploration (not necessary in general, means potentially unupdated state action values).

Summary

- Reinforcement learning setup
- Mathematical formulation: MDP
- Bellman Equation
- Value Iteration Algorithm
- The Q-learning Algorithm

Suggested Re

Textbook: Artificial Intelligence: A Modern
Stuart Russell and Peter Norvig. Pearson,

- Chapter 22